

Payment Ambiguity Decision Layer

Policy & risk reasoning for unresolved payment states · Aastha Narula · v1.9

CHANGED SINCE V1.7

- **Customer trust removed from the scoring formula.** It was a fraud-adjacent judgment of the person, not a fact about the ambiguity. Replaced with `delivery_status` — was the content already delivered when ambiguity was detected — a genuine blast-radius fact. Weights redistributed: `order_value` 0.30→0.35, `payment_method` 0.15 unchanged, `industry_urgency` 0.20→0.25, `trust` 0.35→`delivery_status` 0.25.
- **Customer intent arbitration — actually implemented now.** `confirmed_debited` no longer defaults to a refund. It defaults to `proceed_order_confirmed` (the order just proceeds — the common case), and only refunds when the customer's own cancellation intent is confirmed, arbitrated with the same weak-signal discipline as debit status.
- **Human-readable reasoning trace — implemented.** Every decision now returns a plain-language explanation built from the same data used to decide, persisted alongside the transaction and shown in the admin UI.
- **Escalation ladder refinement beyond original spec:** a confirmed debit status now bypasses the 5-minute fast-path wait entirely — a settled fact doesn't need a buffer to "settle further."
- **Trust-interaction penalty (§8b) removed** along with the trust dimension it depended on — no longer applicable.
- **Known open item, newly surfaced:** the `provisional_access` label reads naturally for not-yet-delivered goods but is a misnomer for already-delivered content in the same risk band — behavior is still correct, naming isn't. Flagged for a v2 refinement, not fixed here.

1 Problem statement

Payment status-check APIs resolve the majority of transaction outcomes, but three gaps persist even at mature PSPs: status-check APIs can themselves time out or return PENDING indefinitely with no guaranteed SLA; gateway webhook, bank/NPCI status, and settlement file can disagree with each other; and refund/reversal lag is real even after a final status is known. The architecture required to detect ambiguity — state machines, idempotency keys, reconciliation jobs — is industry-standard and well solved. What isn't standardized, and differs by company, is the **policy layer**: how long to wait, what to show the customer, how to arbitrate conflicting signals, and what action to take when the SLA window closes without resolution.

This PRD scopes that policy/decision layer — not a payments gateway, and not a claim to have solved payment ambiguity architecturally.

2 Goals

- Design and demonstrate a working risk-based decision policy for transactions stuck in an ambiguous payment state.
- Make the decision explainable: every action traces back to a human-readable reasoning chain, not a black-box score.
- Demonstrate that the same transaction can resolve differently under different merchant/industry policy configurations.
- Build using free-tier tooling (Next.js, Supabase, Vercel) with fully mocked payment providers — no real gateway integration.

Non-goals

- Not building a real payment gateway or processing real money.
- Not claiming to have solved payment ambiguity at an architectural level.
- Not implementing ML-based fraud scoring in v1 (see §8).

3 Who this is for

Primary: a portfolio artifact for AI/payments PM roles, demonstrating product judgment on risk policy design. **Secondary** (in-world framing): a merchant/platform ops team that needs a configurable, auditable policy for resolving ambiguous transactions without manual intervention on every case.

4 Decision dimensions

Each dimension answers the same underlying question: *given we're uncertain, how costly or urgent would it be to get this wrong?* None of them ask anything about who the customer is.

Order value

blast radius

Cost of a wrong auto-resolution — in either direction — scales with value. Higher value warrants more caution.

Delivery status

blast radius

Already-delivered digital content can't be un-delivered. Getting the decision wrong costs more when the mistake can't be clawed back.

Industry / inventory sensitivity

cost of delay

Travel, ticketing, and food delivery have time-locked inventory — ambiguity has a real-time cost. Retail and digital goods tolerate a longer hold with little downside.

Payment method reliability

resolution speed

UPI/NPCI ack-nack is comparatively fast and reliable; netbanking is slower and less predictable — a proxy for how quickly this rail's ambiguity tends to resolve on its own.

Merchant risk appetite

policy knob

Some merchants prioritize never risking a double-give; others prioritize experience. Configurable per merchant, not global (see §6a — specified, not yet built).

5 Risk scoring model

Each dimension is normalized to a 0–1 risk contribution, then combined via a weighted sum.

```
risk_score = 0.35 × order_value_risk
            + 0.25 × delivery_status_risk
            + 0.15 × payment_method_risk
            + 0.25 × industry_urgency_risk
```

Why trust was removed (v1.9 revision): the original weighting included `trust_risk` (customer new/returning, weight 0.35) as a fraud-adjacent proxy. On review, this doesn't belong here — it answers "how much do we trust this person," not "how costly is it to guess wrong while uncertain," which is the only question this engine is built to answer. Trust may legitimately matter to a *separate* fraud system someday (device signals, velocity, dispute history — real fraud evidence, not account age) — but this engine is now deliberately blind to who the customer is, and reasons only about the transaction's own blast radius and urgency.

These remain policy weights, not statistically calibrated probabilities — simple enough to explain to a support or risk team without a model card. The v2 path is calibrating against historical ambiguity outcomes once labeled data exists.

Four decision bands, plus a borderline flag

SCORE RANGE	ACTION	FRICTION
< 0.30	Branches by <code>debit_status</code> & <code>intent</code> (\$5a)	None — resolve immediately
0.30 – 0.449	<code>provisional_access</code>	None — reversible grant
0.45 – 0.599	<code>provisional_access_stepup_required</code>	OTP-style re-verification
≥ 0.60	<code>hold_manual_review</code>	Full human review

Scores within ± 0.03 of any cutoff are marked `borderline: true`. The action still follows the strict cutoff — kept deterministic and auditable — but borderline decisions are flagged for manual spot-check sampling rather than trusted at the same confidence as a clear-cut score.

5aDebit status & customer intent — two separate axes from risk

Risk score answers *"how cautious should we be?"* It does not answer *"was money actually taken?"* or *"did the customer still want this?"* — two further independent questions, each resolved by its own arbitration.

debit_status — confirmed_debited / confirmed_not_debited / unknown — is sourced from the highest-trust signal available (settlement file > bank status API > gateway webhook > client app state).

Corrected default (v1.9): a confirmed_debited result no longer defaults to a refund. It defaults to proceed_order_confirmed — the order just proceeds normally, since a successfully confirmed payment is the common case, not an exception. Refund is the exception, triggered only when customer_intent resolves to confirmed_cancel — an explicit cancel action, arbitrated with the same weak-signal discipline as §5b: a passive signal alone (tab closed, app backgrounded) is too noisy to trust and does not confirm cancellation on its own.

DEBIT_STATUS	INTENT	ACTION	WHY
confirmed_debited	no cancel signal	proceed_order_confirmed	Money taken, customer still wants it — order proceeds.
confirmed_debited	confirmed_cancel	refund_confirmed_debit	Money taken, but the customer explicitly withdrew — refund it.
confirmed_not_debited	—	release_hold_no_action	Nothing taken — close the attempt, no money to move.
unknown	—	refund_precautionary	Status couldn't be confirmed in time — treat conservatively.

This only branches within the lowest risk band — above 0.30, the risk band decides the action regardless of debit status or intent (see §8b for why this is not the same thing as doubting the signal's accuracy).

5b Weak-signal corroboration gate

Client-side app state alone can never confirm a result — for debit status *or* for cancellation intent. State is too easy to be wrong: stale UI, dropped network mid-update, spoofable client state. A report only counts as confirmed if it comes from a medium-or-higher trust source. A lone weak signal resolves to unknown / `no_cancel_signal` pending a more trustworthy source. Implemented and covered by automated tests for both arbitration paths.

5c Action reference — what each outcome actually means

Eight possible actions, each answering a different combination of "how much do we know" and "how much is at stake." Grouped below in the order a transaction would actually move through them.

No decision yet

continue_polling

\$7, < 5 min, unresolved

Nothing has been decided — the system is deliberately still waiting. Most ambiguous transactions resolve themselves within this window through the normal, non-escalated path, so acting before it closes would mean spending decision-engine effort on cases that were about to sort themselves out anyway.

Low risk band (< 0.30) — the money question gets a real answer

Below the 0.30 cutoff, guessing wrong is cheap enough that the system resolves the transaction outright, branching on what's actually known about the money and the customer's intent.

proceed_order_confirmed

score 0.215, ₹200 example

The default for a confirmed debit. Money was taken, and nothing suggests the customer wanted out — so the order simply proceeds, the same way it would if the payment had never been ambiguous in the first place. This used to default to a refund; that was backwards. A successfully confirmed payment is the ordinary case, not an exception that needs undoing.

refund_confirmed_debit

score 0.215, cancellation confirmed

The exception to the row above: money was taken, *and* the customer's own intent to cancel was separately confirmed (an explicit cancel action — not just a closed tab or backgrounded app, which alone proves nothing). Only then does a real refund get issued.

release_hold_no_action

score 0.215, not debited

The bank confirmed nothing was taken. There's no money to move — the only "action" is administrative: close out the stuck attempt and let the customer try again if they want to. Contrast with the two rows above, where money genuinely changed hands and has to be accounted for.

refund_precautionary

score 0.215, status unknown

Nobody confirmed anything — no source gave a trustworthy answer in time. Rather than leave a low-stakes customer stuck indefinitely, the system refunds/releases as a precaution, explicitly accepting the risk of an occasional double-give if settlement later reveals a debit did happen. Worth the trade at this blast radius; not worth it at a higher one.

Medium and above (≥ 0.30) — the money question stops mattering

Past this point, debit_status and cancellation intent no longer change the outcome (see §8b) — the transaction's own blast radius takes over as the only input that matters.

provisional_access

0.30–0.449, score 0.36 example

The system lets the transaction proceed as if it succeeded, but keeps it flagged reversible rather than finalized — no added friction, but nothing about the money is locked in either. This is the first rung of caution: hedge, but don't yet slow the customer down.

provisional_access_stepup_required

0.45–0.599, score 0.465 example

Same idea as above, with one more gate: the customer has to clear an OTP-style re-verification before access is granted. Automated friction — cheaper than a human review queue, more resistant than a free pass. This tier only exists because an earlier single wide "medium" band lumped a mildly-elevated case in with one much closer to review-worthy; splitting it let friction scale with risk instead of jumping straight to a hold.

hold_manual_review

≥ 0.60 , score 0.72 example

The system declines to decide. Not because it doubts any signal — a confirmed debit is still fully trusted here — but because the combination of stakes and irreversibility is high enough that no automated rule should be the one finalizing it alone. A person looks at it instead. The same logic as a bank requiring a second signature on a large wire transfer: not distrust of the first approver, just a checkpoint proportional to the size of the action.

Known naming gap: provisional_access reads naturally for not-yet-delivered goods, where access is genuinely being granted. For already-delivered content landing in the same band, the behavior is still correct (hedge, don't finalize) but there's nothing left to "grant" — the label is a minor misnomer there. Tracked as a v2 naming refinement, not fixed in v1.9.

6 Scenario walkthrough

All figures below are taken directly from running the code (60 automated assertions across arbitration, scoring, intent, and resolver), not hand-calculated.

SCENARIO	VALUE / DELIVERY / METHOD	ELAPSED	SCORE	ACTION
Grocery order, signal not in yet	₹200 / not delivered / UPI	2 min	—	continue_polling
Same order, debit confirmed early	₹200 / not delivered / UPI	2 min	0.215	proceed_order_confirmed
Same order, past fast-path	₹200 / not delivered / UPI	6 min	0.215	proceed_order_confirmed
Same order, customer cancelled	₹200 / not delivered / UPI	6 min	0.215	refund_confirmed_debit
Same order, confirmed NOT debited	₹200 / not delivered / UPI	6 min	0.215	release_hold_no_action
Same order, nothing confirmed	₹200 / not delivered / UPI	6 min	0.215	refund_precautionary
Digital subscription, content delivered	₹500 / delivered / Card	6 min	0.36	provisional_access
Same order, not yet delivered	₹500 / not delivered / Card	6 min	0.21	proceed_order_confirmed
Food delivery, medium-high (borderline)	₹8,000 / not delivered / UPI	6 min	0.465*	provisional_access_stepup_required
Borderline just under 0.30	₹4,000 / not delivered / Card	6 min	0.28*	proceed_order_confirmed

SCENARIO	VALUE / DELIVERY / METHOD	ELAPSED	SCORE	ACTION
High value, delivered, source conflict	₹40,000 / delivered / Netbanking	6 min	0.72	hold_manual_review
Weak signal only (client state alone)	₹300 / not delivered / UPI	6 min	0.215	refund_precautionary
Forced resolution at deadline	₹600 / not delivered / Wallet	30 min	0.305*	provisional_access

* borderline: true — within 0.03 of a cutoff, flagged for spot-check sampling.

6a Same transaction, different merchant policy

The same ₹8,000 transaction (score 0.465) resolves differently depending on where a merchant sets its step-up threshold:

MERCHANT PROFILE	STEP-UP THRESHOLD	RESULTING ACTION
Experience-first (e.g. a travel OTA)	Raised to 0.55	provisional_access — under the raised threshold
Loss-averse (e.g. high-ticket electronics)	Lowered to 0.40	provisional_access_stepup_required — over the lowered threshold

risk_score itself never changes — it's a fact about the transaction. What's configurable is where the band boundaries sit. `specified, not built` — RISK_THRESHOLDS is currently one global constant, not yet parameterized per merchant.

6b Three action classes

One product principle stays sharp by keeping these separate: **retry status checks aggressively; retry the original payment only after it is conclusively confirmed not debited.**

CLASS	WHAT MOVES	PURPOSE
Status-retry	<code>continue_polling</code> — the status check itself	Determine payment state — safe to retry aggressively, no money moves.
Payment retry	A new payment, new idempotency key	Only offered after <code>confirmed_not_debited</code> — prevents a double-debit.
State propagation	Backend/order state, not a user action	<code>proceed_order_confirmed</code> → order marked paid. <code>confirmed_not_debited</code> → retry becomes available.

Known gap, not yet modeled: `proceed_order_confirmed` assumes the order can still be fulfilled. Fulfillability is a separate fact from `debit_status` and isn't tracked yet — flagged as an open item, not assumed away.

7 Escalation ladder

Time since ambiguity was first detected drives when a decision must be made — independent of risk score, with one exception noted below.

WINDOW	BEHAVIOR
< 5 min, status unresolved	<code>continue_polling</code> — no decision made yet.
< 5 min, status already confirmed	<code>proceed</code> immediately — a confirmed fact doesn't need a buffer to settle further. (v1.9 refinement, beyond the original spec.)
5–30 min	Marked AMBIGUOUS. Order locked; idempotency key blocks any retry.
30 min+	Forced resolution. Risk-based action applied even if the score is borderline.

Design principle, not yet implemented: the 30-minute deadline should vary by industry (food delivery needs ~10 min; digital subscriptions could tolerate longer) — LADDER. `forceResolutionAfterMin` is currently one global constant.

8 Why rule-based, not ML (v1)

- **Explainability is required.** Holding or refunding a customer's money needs a clean causal explanation for support, disputes, and audit review.
- **No training data exists yet.** Rules encode expert judgment directly until enough labeled outcomes accumulate.
- **Rules are adjustable in real time.** A wrong threshold is a one-line config change; a wrong ML boundary needs retraining.

8bSource trust vs. risk-based caution

These are two different mechanisms that are easy to conflate — worth being precise about, since they answer genuinely different questions:

	SOURCE TRUST	RISK-BASED CAUTION
Question	Which signal do we believe when sources disagree?	How much blast radius does this transaction carry?
Where	<code>arbitration.ts</code>	<code>resolver.ts</code> (Stage 4)
Resolved by the time...	<code>debit_status</code> reaches Stage 4	The action is chosen

A `hold_manual_review` outcome alongside a *confirmed* debit is not the system doubting the bank — settlement file didn't contradict it, nothing is in dispute. It means the transaction's stakes (order value, delivery status) are high enough that no rule should auto-finalize it alone, the same reason a large wire transfer needs a second signature regardless of how confident the first approver was.

9 Explicit tradeoffs accepted

- Auto-confirming fast reduces friction but risks a payment that later settles differently than expected (merchant eats reconciliation cost).
- Holding longer protects against wrongful resolutions but increases cart abandonment and support load.
- Granting provisional access reduces friction but creates exposure if the blast-radius estimate is wrong.

10 Extensibility requirement

New decision parameters should be addable via configuration — a parameter registry entry plus a policy rule — without modifying the core decision engine. v2 refactor planned once v1's logic is validated against enough real scenarios, deliberately sequenced so extensibility is proven necessary by friction, not designed speculatively ahead of a working core.

11 Success criteria for v1

done

Given a mocked ambiguous transaction, the system returns an action with a human-readable reasoning trace.

done

The escalation ladder correctly gates decisions by time-in-ambiguity, boundary-tested.

done

Identical risk scores, different `debit_status` values, resolve to different correctly-justified actions.

done

`debit_status` derived from disagreeing mock sources via trust-ranked arbitration, not a given input.

done

Customer intent arbitrated with the same weak-signal discipline; `confirmed_debited` correctly defaults to proceeding, not refunding.

done

Borderline scores near a cutoff flagged for lower-confidence handling.

done

60 automated assertions across arbitration, intent, scoring, and resolver — each decision individually justified against \$4.

11cWhere v1 stops

QUESTION	COMPONENT	STATUS
Should we keep waiting?	Escalation ladder	implemented, tested
Which payment signal do we trust?	Signal arbitration	implemented, tested
Was money taken?	debit_status	implemented, tested
Did the customer withdraw intent?	Intent arbitration	implemented, tested
How risky is the transaction?	Risk score	implemented, tested
What should we actually do?	Resolution action	implemented, tested
Why did we decide this?	Reasoning trace	implemented, tested
Is "provisional access" the right label for delivered goods?	Action naming	known gap, v2
Can we actually fulfil the order?	Fulfillment check	open – v2
What if settlement contradicts an earlier decision?	Reconciliation workflow	open – v2

12 Out of scope for v1

- Real payment gateway integration and live data sourcing.
- ML-based fraud scoring.
- Feature-provider abstraction layer for external data sources.
- Multi-merchant policy configuration UI.
- Structured hold_manual_review handoff payload (top risk drivers, suggested action, confidence) — specified in earlier drafts, not yet built.